

2025



Hoptech.

LYNK0

DOCUMENTACIÓN DE LAS APIS
RONALD CARGUA

HOPTECH

Tabla de contenido

1. OBJETIVO.....	3
2. ALCANCE	3
3. ALOJAMIENTO DE LAS APIS.....	3
4. DATOS DE ENTRADA Y SALIDA DE TRAMAS ENCRYPTADAS.....	4
5. SERVICIOS	11
5.1 CLIENTES.....	11
Aplicación: Nomina – GetClients	11
Aplicación: Nomina – ActualizaCliente.....	12
Aplicación: Bankingly – GetClientsByDocument	14
Aplicación: Bankingly – GetAccountOfficerByClient.....	15
Aplicación: Bankingly – ValidateNewUser	16
Aplicación: Bankingly – GetUserInfoInformation	17
Aplicación: Bankingly – UpdateUserInfoInformation	18
5.2 PRODUCTOS.....	19
Aplicación: Bankingly – GetProducts.....	19
Aplicación: Bankingly – GetProductsConsolidatedPosition	20
Aplicación: Bankingly – GetExchangeRates.....	20
Aplicación: Bankingly – GetProductBankStatements.....	21
5.3 CUENTAS	23
Aplicación: Bankingly – GetAccountDetails.....	23
Aplicación: Bankingly – GetAccountMovements.....	24
Aplicación: Bankingly – GetTransactionVoucher.....	25
Aplicación: Bankingly – GetMobileNumberAccounts.....	25
5.4 PRÉSTAMOS	27
Aplicación: Bankingly – GetLoan.....	27
Aplicación: Bankingly – GetLoanFees	28
Aplicación: Bankingly – GetLoanPayments	29
Aplicación: Bankingly – GetLoanRates	30
5.5 TRANSACCIONES	30
Aplicación: Bankingly – InsertTransaction.....	31
Aplicación: Bankingly – GetExchangeRateTransaction.....	32
Aplicación: Bankingly – GetTransactionCost	32
Aplicación: Bankingly – InsertMassiveTransaction	33
5.6 AVAL BURÓ – Historial Crediticio.....	34
Aplicación: Aval Buró – Consulta de Historial Crediticio.....	34
5.7 REGISTRO CIVIL.....	35
Aplicación: Registro Civil – Consulta de Personas.....	35
5.8 MENSAJERÍA.....	36
Aplicación: Mensajería – Enviar Mensajes de Texto y Correos Electrónicos	36

5.9 COONECTA	38
Aplicación: Consulta de Clientes por número de Cuenta.....	38

1. OBJETIVO

El propósito fundamental del presente manual de documentación de APIs es proveer a desarrolladores, usuarios y equipos técnicos una referencia completa y precisa acerca del funcionamiento, la estructura y la utilización de las APIs ofrecidas para la plataforma de Multicomm (Satélites). A través de esta documentación, se busca facilitar la exitosa integración de las APIs en diversas aplicaciones y sistemas,

proporcionando información detallada acerca de los *endpoints*, los parámetros de solicitud, las respuestas anticipadas y los posibles escenarios de uso. Este manual persigue los siguientes propósitos:

- Ofrecer una descripción general de las APIs disponibles, resaltando sus funcionalidades.
- Detallar los pasos necesarios para adquirir credenciales de API y establecer un proceso de autenticación adecuado.
- Explicar el proceso de construcción de solicitudes válidas, incluyendo la estructura de los *endpoints* y los parámetros requeridos.
- Aclarar los tipos de respuestas que cabe esperar por parte de las APIs y brindar orientación en la interpretación de los códigos de estado correspondientes.
- Presentar ejemplos prácticos y casos de uso con el fin de ilustrar una implementación exitosa de las APIs.

2. ALCANCE

Esta documentación de APIs está diseñada exclusivamente para el consumo y referencia de los equipos de desarrollo de HOPTECH y COAC LUZ DEL VALLE. El contenido de este manual está destinado a proporcionar información detallada y específica sobre las APIs que se encuentran bajo la Colaboración conjunta entre ambas partes. Los detalles cubiertos incluyen descripciones de *endpoints*, parámetros de solicitud, estructura de respuestas y casos de uso relevantes para los proyectos en curso. Cabe destacar que esta documentación es confidencial y no debe ser compartida con terceros ni utilizada para fines ajenos a la colaboración entre HOPTECH y COAC LUZ DEL VALLE .

3. ALOJAMIENTO DE LAS APIS

Al utilizar diversas APIs, es importante considerar las URL base correspondientes a los entornos de pruebas y producción.

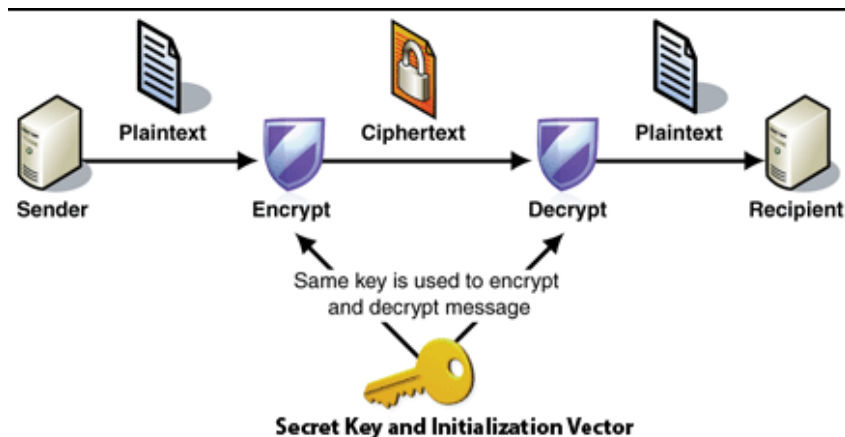
- **URL del ambiente de pruebas:**
 - <http://lynko-test.hoptech.dev/>
 - <https://lynko-test.hoptech.dev/>
- **URL del ambiente de producción:**
 - <http://lynko.hoptech.dev/>
 - <http://lynko.hoptech.dev/>

4. DATOS DE ENTRADA Y SALIDA DE TRAMAS ENCRYPTADAS

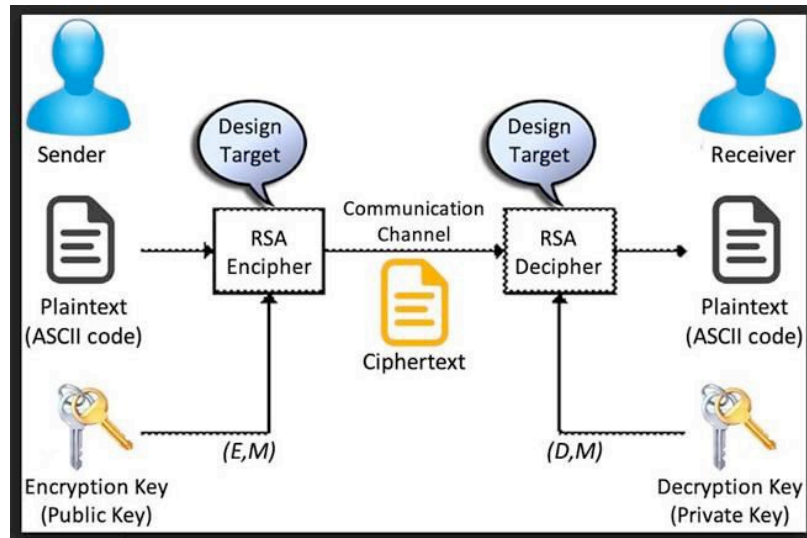
Para los siguientes Web Services existen dos procesos que se deben realizar antes de poder comunicarse al Multicomm que son encriptar y desencriptación.

Los métodos de encriptación a utilizar son AES(256) y RSA(4096).

- AES
 - Key Size in Bits: 256
 - Mode : CBC
 - Initialization Vector Length: 16
 - Secret Key Length: 32
- RSA
 - Key Size in Bits: 4096
 - RSA Ciphers: RSA/NONE/OAEPWithSHA1AndMGF1Padding
- AES es un algoritmo de cifrado que mediante el uso de una clave secreta y un vector de inicialización compartidos, permiten encriptar un mensaje/trama.
- Para encriptar el mensaje JSON ocupamos AES256.



- RSA es un algoritmo de cifrado que utiliza un par de llaves (Pública y Privada) para su funcionamiento. Para la encriptación se emplea la llave Pública, mientras que para la descryptación se utiliza la llave privada.
- Para encriptar tanto la clave secreta como el vector de inicialización ocupamos RSA utilizando la llave pública.



1. Proceso de Encriptación Satellite-Lynko

Satellite encriptará la información utilizando el algoritmo AES256, mientras que la clave secreta y el vector de inicialización se encriptarán con llave pública de RSA localizada en (/ida/publicKey.pem)

Por ejemplo cuando el Satellite desee consultar la información de un cliente utilizando el servicio ConsultarClienteCodigo lo hará de la siguiente manera:

- Generar un JSON con la información a consultar

```
{
  " cliCodigoCore" : "255"
}
```

- Encriptar con AES el JSON, para lo cual se debe generar aleatoriamente un secretKey y un IV (Vector de Inicialización). Con estos 2 datos se procede a encriptar el JSON. Como resultado se obtiene un texto encriptado.

Ejemplo:

Clave secreta: 12345678901234567890123456789012

Vector de inicialización: 1234567890123456

Texto encriptado:

8JUjhJfF+ZcRXm1IeSGbkjtDo3LHkymO

- c) Encriptar con la llave pública de RSA localizada en (/ida/publicKey.pem), la clave secreta y el vector de inicialización.

Resultado

Clave secreta:

ZGRJMGZfnR4V7cm2y53tP1UxhrNOCogRtfOYJf+NJupdOG09VHJ1uq0XZsAnXWf
TIZcShjrk9HF3ZKjLjfvnu54NnvxHWCrs5qxGexPfV\zCZK3YJs4VKuKSHI2pMfvtOz
FfZ0CnPaAGkLHr9yyNnhahMg2j6ElsiRw7qwfOGYpz\paPtAAQ5N1+0SaiA59c++9Z
R+ =

Vector de inicialización:

LWe8ABLqajCVCapfEXIXzjOyy0fc8xDEm1pOJkJvMpVpL\micujA2vdZHwQoLnYL
x44Z5+EZVYZak8oLNZTX2qGkfUUbPKwrAukATykkZ7FAuOy7N+mIle6FT7AifaD
X0tbYzHQaWE\FzDKDVIahdJBTOQXwRhbsznSmrpyTz0un1DgopjoAs4UHyXJOnw
N3jtA0Z9ztYJB03Ez2K8aO6HFJzX5yJ07UPYRzyU7sZlUiZaGfmMVfneUOCPMHim
55GrWC1Wb501lRb5Ch2sewFZHqIk1BYIwGLbgPG09T6HpDgkoOriyd6ibQGp0gGj
WinoL1fGvdG4=

Texto encriptado:

8JUjhJfF+ZcRXm1IeSGbkjtDo3LHkymO

- d) Generar un JSON con la información de encriptación SessionKey = Clave secreta, IV = Vector de inicialización, Data = Texto encriptado

```
{  
    "SessionKey":  
    +IGh7co3Uva96uQKQ97O2I9jLfZ\NL5OJEZc2nGrxNWfqYUMu1c3gjBcNnKM\MV  
    hDi2YYyXcD4LZ+WM1Uh75ymmHvRYbledZMWCuvIL\9nq2QEY5uz6AgVcDied  
    Vslgk+p2tUtOynzNn5sK\FVXMKF1f8VaHWJ0UzgrFNN01\tazCuoDoKv6n0LPCgW  
    XOjSVvR6nA\FKWCmXisB43Nqr5iLLW6wbQiPwHJqYuiranvoR1p6sUB42iBaymgD
```

```
q1wjUE0UCyVwIGTUpcyFdsK+sXOgpAK1fvasB5JV2\HOgsknpNEm+KdUwNmIVk
QGvIqvhtZWCDDu7E=",
```

"IV":

```
"EZ\YZak8oLNZTX2qGkfUUbPKwrAukATykkZ7FAuOy7N+mIle6FT7AifaDX0tbYz
HQaWEVfZDKDVIahdJBTOQXwRhbsznSmrpyTz0un1DgopjoAs4UHyXJOnwN3jtA0
Z9ztYJB03Ez2K8aO6HFJzX5yJ07UPYRzyU7sZlUIiZaGfmMVfneUOCPMHim55GrW
C1Wb501lRb5Ch2sewFZHqIk1BYIwGLbgPG09T6HpDgkoOriyd6ibQGp0gGjWinoL1f
GvdG4=",
```

"Data": "8JUjhJfF+ZcRXm1IeSGbkjtDo3LHkymO"

}

- e) Enviar la información al servicio provisto por el Multicomm.

2. Descriptación Lynko - Satellite

Una vez recibido esta información del lado del Multicomm, primero se deberá descriptar el contenido del campo "IV" y "SessionKey" del JSON utilizando la llave privada localizada en (/ida/privateKey.pem); a continuación se deberá descriptara el campo "Data" con el algoritmo TripleDES, para este fin se deberá utilizar el IV (Vector de inicialización) y SessionKey (Clave secreta) descriptados con anterioridad.

Si la descriptación fue exitosa se obtendrá el mensaje original en formato JSON.

```
{
```

```
" cliCodigoCore" : "255"
```

```
}
```

Y con esta información, se podrá continuar con su procesamiento.

3. Encriptación Lynko - Satellite

Lynko encriptará la información utilizando el algoritmo AES256, mientras que la clave secreta y el vector de inicialización se encriptarán con llave pública de RSA localizada en (/vuelta/publicKey.pem)

Para retornar la información, se lo hará de la siguiente manera:

- a. El Banco deberá armar el JSON conforme la información enviada en el archivo “Definición de los WebServices”, similar al siguiente:

```
{
  "cliCodigoCore": "255",
  "estadoTransaccion": true,
  "cliTipoPersona": "1",
  "cliApellido1": "OLMEDO",
  "cliApellido2": "JIMENEZ",
  "cliNombre1": "MARIO",
  "cliNombre2": "MANUEL",
  "cliNombreCompleto": "OLMEDO JIMENEZ MARIO MANUEL",
  "cliRazonSocial": "",
  "cliSexo": "M",
  "cliLugarNacimiento": "SAN FRANCISCO",
  "cliFechaNacimiento": "1980-01-01",
  "cliObservaciones": "NINGUNA",
  "cliEmail": "alidar@Banco.org.ec",
  "cliEstado": "A",
  "cliCelular": "0981003824",
  "cliIdentificacion": "",
  "errorCodigo": "0",
  "errorTipo": "",
  "errorDescripcion": ""
}
```

- b. Encriptar con AES256 el JSON, para lo cual se debe generar aleatoriamente un secretKey y un IV (Vector de Inicialización). Con estos 2 datos se procede a encriptar el JSON. Como resultado se obtiene un texto encriptado.

Ejemplo:

Clave secreta: 23987456

Vector de inicialización: 09346723

Texto encriptado:

YI6ThR7TG\Tp+sfkX1GY6Ns89PohyqA86hO07uxp2bsL1JMzFVjniFDN49
FZCvaZWY2QfOIDAswUX1bEXWlbsbWs2+ROQl67IFIf5wv9V9KT3nfSK
HQKBgic8cFRvBTWxxlRM8tfZ6IanzR4pu2b68XRLjiXuBhPRBg4JueMK5jl
lpV9cZIGt4D0dvuw2H8W0ItpjznC2KATH6HFSyWNpkP8jzdPuOm2hM+tiC
RNY6Jh8qV9T5Ud\wckeXcw\8eFyAGximwdcz2CNtY56d45TT+tNhJHPFfr
0YOF\Z7RbtBwUnQyRznyH2tmm6

- c. Encriptar con la llave pública de RSA localizada en (/vuelta/publicKey.pem), la clave secreta y el vector de inicialización.

Clave secreta:

F1N2UOZL3MYvkKpPZJdcKCahX4Ssr7NkVlIjp6Qv8fB9m9uUqKggJtuoAw
D1rv393JnxN9SJGjaraFFPzOQEgrNQ6rvue2Gji5N1MuoeIXJOBLeqInjW5n
Dw4jFjWUPPRoS8VWTURNegGE62qvp3wc3Z9sZk7jyqt0hxOvLS9UA8NB
FQMQUf+NZPoa4BH7i5mbQmrM2j5tfA8l9PNm05H5hmF8urBcupky1uy6A
e8rIcPjKoUoniS8E0aWg8eaH7VANnQrI=

Vector de inicialización:

\rm4w+UBzqlsfRXhXCuIRY4mVZsh\O29DGluq03K9OujXOWHjSMp4dos
1xu0pbHuoUdWKzmfERTauuqWWcXOaMjqn4EWotGZfDFI7ILYwntYflaY
Wp4hjHj8\rzL6V\WKV12LLxmh0u0iWX\Usek1pMynm9AJcYs2ZalTG0x
ybD524eRl5YNZEu4joeTB9A8AoyCGmTIYORbb04PCtRU3DF9XrexoYfQi
d3mCl+HZPraDwJ5EuCWoc1a+TLw89FVMnH5rVPNkG7eksdbqdS\iphfX9
fLxJV3h7f6QlS8tTSHZovojvyUbLL\VIWSqXkQr4O+iKHz393dObjVNdrEiE
yh6XUhezBjMghxKMe2gxkvQT6lSfOipfl4xizCFIN9gRluEciCiI=

Texto encriptado:

I6ThR7TG\Tp+sfkX1GY6Ns89PohyqA86hO07uxp2bsL1JMzFVjniFDN49F
ZCvaZWY2QfOIDAswUX1bEXWlbsbWs2+ROQl67IFIf5wv9V9KT3nfSKH
QKBgic8cFRvBTWxxlRM8tfZ6IanzR4pu2b68XRLjiXuBhPRBg4JueMK5jllp
V9cZIGt4D0dvuw2H8W0ItpjznC2KATH6HFSyWNpkP8jzdPuOm2hM+tiCR

```
NY6Jh8qV9T5Ud\wckeXcw\8eFyAGximwdcz2CNtY56d45TT+tNhJHPFfr0
YOF\Z7RbtBwUnQyRznyH2tmm6
```

- d. Generar un JSON con la información de encriptación SessionKey = Clave secreta, IV = Vector de inicialización, Data = Texto encriptado

```
{
  "SessionKey": "
Vlljp6Qv8fB9m9uUqKggJtuoAwD1rv393JnxN9SJGjaraFFPzOQEgrNQ6rvue
2Gji5N1MuoeIXJOBLeqInjW5nDw4jffWUPPRoS8VWTURNegGE62qvp3w
c3Z9sZk7jyqt0hxOvLS9UA8NBFQMQUf+NZPoa4BH7i5mbQmrM2j5tfA8l9
PNm05H5hmF8urBcupky1uy6Ae8rlcPjKoUoniS8E0aWg8eaH7VANnQrI=",
  "IV": "
\WKV12LLxmh0u0iWX\Usek1pMynm9AJcYs2ZalTG0xybD524eRl5YNZE
u4joeTB9A8AoyCGmTIYORbb04PcTrU3DF9XrexoYfQid3mCl+HZPraDwJ
5EuCWoc1a+TLw89FVMnH5rVPNkG7eksdbqdS\iphfX9fLxJV3h7f6QlS8tT
SHZovojvyUbLL\WIWSqXkQr4O+iKH393dObjVNdrEiEyh6XUhezBjMghx
KMe2gxkvQT6lSfOipfl4xizCFIN9gRluEciCiI=",
  "Data": "
RBg4JueMK5jllpV9cZIGt4D0dvuw2H8W0ItpjznC2KATH6HFSyWNpkP8jz
dPuOm2hM+tiCRNY6Jh8qV9T5Ud\wckeXcw\8eFyAGximwdcz2CNtY56d
45TT+tNhJHPFfr0YOF\Z7RbtBwUnQyRznyH2tmm6"
}
```

- e. Este JSON será la respuesta enviada por el servicio ConsultarClienteCodigo
- f. Satelite procederá a desencriptar el JSON utilizando para el campo “IV” y “SessionKey” la llave privada localizada en (/vuelta/privateKey.pem); a continuación se deberá desencriptara el campo “Data” con el algoritmo AES, para este fin se deberá utilizar el IV (Vector de inicialización) y SessionKey (Clave secreta) desencriptados con anterioridad.

5. SERVICIOS

5.1 CLIENTES

Aplicación: Nomina – GetClients

Descripción: Consulta de cliente por número de identificación.

Path: /Nomina/Clients/GetClients

Método: POST

Cabeceras de la petición

KEY	VALOR
Content-Type	application/json
aplicacioncliente	Nomina
aplicacionservidor	Denarius
nombreservicio	ServiciosClientes
nombreClase	ConsultarClientePorIdentificacion
nombreMetodo	meConsultasClientes

Datos de entrada

CAMPO	DESCRIPCIÓN	TIPO DE DATO
DocumentType	Tipo de documento (CED: Cédula, RUC: RUC, PAS: Pasaporte)	string
DocumentNumber	Número de documento del cliente	string

Cuerpo de la petición

```
{
  "DocumentType": "string",
  "DocumentNumber": "string"
}
```

Respuesta

```
{
  "tipoVivienda": "",
  "codigoTipoVivienda": "",
  "esFuncionario": "",
  "codigoNivelEstudios": "",
  "nivelEstudios": "",
  "esVinculado": "",
  "provincia": "",
  "ciudad": "",
  "direccion": "",
  "estadoCivil": "",
  "descripcionNacionalidad": "",
  "cliCodigoCore": "",
}
```

```
"cliIdentificacion": "",
"cliTipoPersona": "",
"cliApellido1": "",
"cliApellido2": "",
"cliNombre1": "",
"cliNombre2": "",
"cliNombreCompleto": "",
"cliRazonSocial": "",
"cliSexo": "",
"cliLugarNacimiento": "",
"cliFechaNacimiento": "",
"cliFechaConstitucion": "",
"cliObservaciones": "",
"cliEmail": "",
"cliCelular": "",
"cliTelefono": "",
"cliTieneTarjeta": "",
"cliTipoIdentificacion": "",
"estadoTransaccion": "",
"errorCodigo": "",
"errorTipo": "",
"errorDescripcion": "",
"idCelular": ""
}
```

Aplicación: Nomina – ActualizaCliente

Descripción: Consulta de cliente por número de identificación.

Path: /Nomina/Clients/actualizaCliente

Método: POST

Cabeceras de la petición

KEY	VALOR
Content-Type	application/json
aplicacioncliente	Nomina
aplicacionservidor	Denarius
nombreservicio	ServiciosClientes
nombreClase	ConsultarDatosCliente
nombreMetodo	meActualizacion

Datos de entrada

CAMPO	DESCRIPCIÓN	TIPO DE DATO
Actualiza	Campo que se va a actualizar.(Tel: Telefono, Email: Correo Electronico, Dir: Direcciones)	string
IdCliente	Número de id del cliente	string
CampoAntiguo	Objeto del campo antiguo a actualizar dependiendo del	JsonObject

	campo Actualiza	
CampoNuevo	Objeto del campo nuevo a actualizar dependiendo del campo Actualiza	JObject

Cuerpo de la petición

```
{
  "Actualiza": "string",
  "IdCliente": "string",
  "CampoAntiguo": {},
  "CampoNuevo": {}
}

Campo Antiguo
Telefono:
{
  "actual": "true",
  "codigoArea": "0",
  "codigoRegion": "09",
  "extension": "",
  "idTelefono": "494021",
  "numeroTelefono": "93466444",
}

Email:
{
  "email": "ronald_96c@hotmail.com"
}

Direcciones:
{
  "actual": "",
  "callePrincipal": "",
  "calleTransversal": "",
  "idDireccion": "",
  "numeracion": "",
}

Campo Nuevo
Telefono:
{
  "actual": "true",
  "codigoArea": "0",
  "codigoRegion": "09",
  "extension": "",
  "idTelefono": "494021",
  "numeroTelefono": "93466444",
  "observaciones": "",
  "pais": "EC",
  "tipoTelefono": "C",
  "verificado": "true",
  "principal": "true"
}
```

```
Email:
{
    "email": "ronald_96c@hotmail.com"
}
Direcciones:
{
    "actual": "",
    "callePrincipal": "",
    "calleTransversal": "",
    "canton": "",
    "ciudad": "",
    "idDireccion": "",
    "numeracion": "",
    "pais": "",
    "parroquia": "",
    "principal": "",
    "provincia": "",
    "sector": "",
    "tiempoResidencia": "",
    "tipoDireccion": "",
    "verificada": ""
}
```

Respuesta

```
{
    "estadoTransaccion": "",
    "errorCodigo": "",
    "errorTipo": "",
    "errorDescripcion": ""
}
```

Aplicación: Bankingly – GetClientsByDocument

Descripción: Consulta de cliente por número de documento.

Path: /Bankingly/Clients/GetClientsByDocument

Método: POST

Cabeceras de la petición

KEY	VALOR
Content-Type	application/json
Authorization	Bearer {token}

Datos de entrada

CAMPO	DESCRIPCIÓN	TIPO DE DATO
-------	-------------	--------------

TenantId	Identificador de la institución	string
DocumentType	Tipo de documento (Cédula, RUC, etc.)	string
DocumentNumber	Número de documento del cliente	string

Cuerpo de la petición

```
{
  "TenantId": "string",
  "DocumentType": "string",
  "DocumentNumber": "string"
}
```

Datos de salida

CAMPO	DESCRIPCIÓN	TIPO DE DATO
success	Indica si la operación fue exitosa	boolean
clientId	Identificador del cliente	string
fullName	Nombre completo del cliente	string
birthDate	Fecha de nacimiento	date
email	Correo electrónico del cliente	string
phone	Teléfono del cliente	string

Respuesta

```
{
  "GetClientsByDocumentResult": {
    "success": true,
    "clientId": "string",
    "fullName": "string",
    "birthDate": "yyyy-MM-dd",
    "email": "string",
    "phone": "string"
  }
}
```

Aplicación: Bankingly – GetAccountOfficerByClient

Descripción: Obtiene el oficial de cuenta asignado a un cliente.

Path: /Bankingly/Clients/GetAccountOfficerByClient

Método: POST

Cabeceras de la petición

KEY	VALOR
Content-Type	application/json
Authorization	Bearer {token}

Datos de entrada

CAMPO	DESCRIPCIÓN	TIPO DE DATO
TenantId	Identificador de la institución	string
ClientId	Identificador único del cliente	string

Cuerpo de la petición

```
{
  "TenantId": "string",
  "ClientId": "string"
}
```

Datos de salida

CAMPO	DESCRIPCIÓN	TIPO DE DATO
success	Indica si la operación fue exitosa	boolean
officerName	Nombre del oficial asignado	string
email	Correo del oficial	string
phone	Teléfono del oficial	string

Respuesta

```
{
  "GetAccountOfficerByClientResult": {
    "success": true,
    "officerName": "string",
    "email": "string",
    "phone": "string"
  }
}
```

Aplicación: Bankingly – ValidateNewUser

Descripción: Valida la información de un usuario nuevo.

Path: /Bankingly/Clients/ValidateNewUser

Método: POST

Cabeceras de la petición

KEY	VALOR
Content-Type	application/json
Authorization	Bearer {token}

Datos de entrada

CAMPO	DESCRIPCIÓN	TIPO DE DATO
TenantId	Identificador de la institución	string
DocumentNumber	Documento del cliente	string
Email	Correo electrónico del cliente	string
Phone	Teléfono del cliente	string

Cuerpo de la petición

```
{
  "TenantId": "string",
  "DocumentNumber": "string",
  "Email": "string",
  "Phone": "string"
}
```

Datos de salida

CAMPO	DESCRIPCIÓN	TIPO DE DATO
success	Indica si la operación fue exitosa	boolean
isValid	Indica si los datos son válidos	boolean
message	Mensaje de validación	string

Respuesta

```
{
  "ValidateNewUserResult": {
    "success": true,
    "isValid": true,
    "message": "string"
  }
}
```

Aplicación: Bankingly – GetUserInformation

Descripción: Obtiene información detallada de un usuario.

Path: /Bankingly/Clients/GetUserInformation

Método: POST

Datos de entrada

CAMPO	DESCRIPCIÓN	TIPO DE DATO
TenantId	Identificador de la institución	string
UserId	Identificador del usuario	string

Cuerpo de la petición

```
{
  "TenantId": "string",
  "UserId": "string"
}
```

Datos de salida

CAMPO	DESCRIPCIÓN	TIPO DE DATO
success	Indica si fue exitosa	boolean
userId	Identificador del usuario	string
fullName	Nombre completo	string
email	Correo electrónico	string
phone	Teléfono	string
status	Estado del usuario	string

Respuesta

```
{
  "GetUserInformationResult": {
    "success": true,
    "userId": "string",
    "fullName": "string",
    "email": "string",
    "phone": "string",
  }
}
```

```
    "status": "string"
  }
}
```

Aplicación: Bankingly – UpdateUserInformation

Descripción: Actualiza datos de un usuario existente.

Path: /Bankingly/Clients/UpdateUserInformation

Método: POST

Datos de entrada

CAMPO	DESCRIPCIÓN	TIPO DE DATO
TenantId	Identificador de la institución	string
UserId	Identificador del usuario	string
Email	Correo electrónico actualizado	string
Phone	Teléfono actualizado	string

Cuerpo de la petición

```
{
  "TenantId": "string",
  "UserId": "string",
  "Email": "string",
  "Phone": "string"
}
```

Datos de salida

CAMPO	DESCRIPCIÓN	TIPO DE DATO
success	Indica si la operación fue exitosa	boolean
message	Mensaje de confirmación	string

Respuesta

```
{
  "UpdateUserInformationResult": {
    "success": true,
    "message": "Datos actualizados correctamente"
  }
}
```

5.2 PRODUCTOS

Aplicación: Bankingly – GetProducts

Descripción: Lista los productos asociados a un cliente.

Path: /Bankingly/Products/GetProducts

Método: POST

Cabeceras de la petición

KEY	VALOR
Content-Type	application/json
Authorization	Bearer {token}

Datos de entrada

CAMPO	DESCRIPCIÓN	TIPO DE DATO
TenantId	Identificador de la institución	string
ClientId	Identificador único del cliente	string

Cuerpo de la petición

```
{
  "TenantId": "string",
  "ClientId": "string"
}
```

Datos de salida

CAMPO	DESCRIPCIÓN	TIPO DE DATO
success	Indica si fue exitosa	boolean
products	Lista de productos asociados	array
productId	Identificador del producto	string
productType	Tipo de producto (Cuenta, Préstamo, Tarjeta)	string
status	Estado del producto	string

Respuesta

```
{
  "GetProductsResult": {
    "success": true,
    "products": [
      {
        "productId": "string",
        "productType": "string",
        "status": "string"
      }
    ]
  }
}
```

Aplicación: Bankingly – GetProductsConsolidatedPosition

Descripción: Consulta el estado consolidado de los productos de un cliente.

Path: /Bankingly/Products/GetProductsConsolidatedPosition

Método: POST

Cabeceras de la petición

KEY	VALOR
Content-Type	application/json
Authorization	Bearer {token}

Datos de entrada

CAMPO	DESCRIPCIÓN	TIPO DE DATO
TenantId	Identificador de la institución	string
ClientId	Identificador único del cliente	string

Cuerpo de la petición

```
{
  "TenantId": "string",
  "ClientId": "string"
}
```

Datos de salida

CAMPO	DESCRIPCIÓN	TIPO DE DATO
success	Indica si la operación fue exitosa	boolean
totalBalance	Balance total de todos los productos	decimal
products	Lista de productos	array

Respuesta

```
{
  "GetProductsConsolidatedPositionResult": {
    "success": true,
    "totalBalance": "decimal",
    "products": []
  }
}
```

Aplicación: Bankingly – GetExchangeRates

Descripción: Consulta los tipos de cambio disponibles.

Path: /Bankingly/Products/GetExchangeRates

Método: POST

Cabeceras de la petición

KEY	VALOR
Content-Type	application/json
Authorization	Bearer {token}

Datos de entrada

CAMPO	DESCRIPCIÓN	TIPO DE DATO
TenantId	Identificador de la institución	string
CurrencyFrom	Moneda de origen	string
CurrencyTo	Moneda de destino	string

Cuerpo de la petición

```
{
  "TenantId": "string",
  "CurrencyFrom": "string",
  "CurrencyTo": "string"
}
```

Datos de salida

CAMPO	DESCRIPCIÓN	TIPO DE DATO
success	Indica si la operación fue exitosa	boolean
rate	Tipo de cambio actual	decimal
timestamp	Fecha y hora de la cotización	datetime

Respuesta

```
{
  "GetExchangeRatesResult": {
    "success": true,
    "rate": "decimal",
    "timestamp": "yyyy-MM-ddTHH:mm:ss"
  }
}
```

Aplicación: Bankingly – GetProductBankStatements

Descripción: Obtiene los estados de cuenta de un producto.

Path: /Bankingly/Products/GetProductBankStatements

Método: POST

Cabeceras de la petición

KEY	VALOR
Content-Type	application/json
Authorization	Bearer {token}

Datos de entrada

CAMPO	DESCRIPCIÓN	TIPO DE DATO
TenantId	Identificador de la institución	string
ProductId	Identificador del producto	string

Cuerpo de la petición

```
{
  "TenantId": "string",
  "ProductId": "string"
}
```

```
}
```

Datos de salida

CAMPO	DESCRIPCIÓN	TIPO DE DATO
success	Indica si la operación fue exitosa	boolean
statements	Lista de estados de cuenta disponibles	array
statementId	Identificador del estado de cuenta	string
period	Periodo del estado de cuenta (mes, año)	string
balance	Balance reflejado en el estado	decimal

Respuesta

```
{
  "GetProductBankStatementsResult": {
    "success": true,
    "statements": [
      {
        "statementId": "string",
        "period": "string",
        "balance": "decimal"
      }
    ]
  }
}
```

5.3 CUENTAS

Aplicación: Bankingly – GetAccountDetails

Descripción: Obtiene el detalle de una cuenta bancaria.

Path: /Bankingly/Accounts/GetAccountDetails

Método: POST

Cabeceras de la petición

KEY	VALOR
Content-Type	application/json
Authorization	Bearer {token}

Datos de entrada

CAMPO	DESCRIPCIÓN	TIPO DE DATO
TenantId	Identificador de la institución	string
AccountNumber	Número de la cuenta a consultar	string

Cuerpo de la petición

```
{
  "TenantId": "string",
  "AccountNumber": "string"
}
```

Datos de salida

CAMPO	DESCRIPCIÓN	TIPO DE DATO
success	Indica si la operación fue exitosa	boolean
accountNumber	Número de la cuenta	string
accountType	Tipo de cuenta (ahorro/corriente)	string
balance	Saldo actual	decimal
currency	Moneda de la cuenta	string
status	Estado de la cuenta	string

Respuesta

```
{
  "GetAccountDetailsResult": {
    "success": true,
    "accountNumber": "string",
    "accountType": "string",
    "balance": "decimal",
    "currency": "string",
    "status": "string"
  }
}
```


Aplicación: Bankingly – GetAccountMovements

Descripción: Obtiene los movimientos de una cuenta bancaria.

Path: /Bankingly/Accounts/GetAccountMovements

Método: POST

Cabeceras de la petición

KEY	VALOR
Content-Type	application/json
Authorization	Bearer {token}

Datos de entrada

CAMPO	DESCRIPCIÓN	TIPO DE DATO
TenantId	Identificador de la institución	string
AccountNumber	Número de cuenta a consultar	string
FromDate	Fecha de inicio (yyyy-MM-dd)	date
ToDate	Fecha de fin (yyyy-MM-dd)	date

Cuerpo de la petición

```
{
  "TenantId": "string",
  "AccountNumber": "string",
  "FromDate": "yyyy-MM-dd",
  "ToDate": "yyyy-MM-dd"
}
```

Datos de salida

CAMPO	DESCRIPCIÓN	TIPO DE DATO
success	Indica si la operación fue exitosa	boolean
movements	Lista de movimientos	array
date	Fecha del movimiento	date
description	Descripción del movimiento	string
amount	Monto del movimiento	decimal
balance	Saldo después del movimiento	decimal

Respuesta

```
{
  "GetAccountMovementsResult": {
    "success": true,
    "movements": [
      {
        "date": "yyyy-MM-dd",
        "description": "string",
        "amount": "decimal",
        "balance": "decimal"
      }
    ]
  }
}
```

Aplicación: Bankingly – GetTransactionVoucher

Descripción: Obtiene el comprobante de una transacción de cuenta.

Path: /Bankingly/Accounts/GetTransactionVoucher

Método: POST

Cabeceras de la petición

KEY	VALOR
Content-Type	application/json
Authorization	Bearer {token}

Datos de entrada

CAMPO	DESCRIPCIÓN	TIPO DE DATO
TenantId	Identificador de la institución	string
TransactionId	Identificador único de la transacción	string

Cuerpo de la petición

```
{
  "TenantId": "string",
  "TransactionId": "string"
}
```

Datos de salida

CAMPO	DESCRIPCIÓN	TIPO DE DATO
success	Indica si fue exitosa	boolean
transactionId	Identificador de la transacción	string
voucher	Comprobante en base64	string

Respuesta

```
{
  "GetTransactionVoucherResult": {
    "success": true,
    "transactionId": "string",
    "voucher": "base64"
  }
}
```

Aplicación: Bankingly – GetMobileNumberAccounts

Descripción: Consulta las cuentas asociadas a un número de teléfono móvil.

Path: /Bankingly/Accounts/GetMobileNumberAccounts

Método: POST

Cabeceras de la petición

KEY	VALOR
Content-Type	application/json
Authorization	Bearer {token}

Datos de entrada

CAMPO	DESCRIPCIÓN	TIPO DE DATO
TenantId	Identificador de la institución	string
MobileNumber	Número de teléfono móvil	string

Cuerpo de la petición

```
{
  "TenantId": "string",
  "MobileNumber": "string"
}
```

Datos de salida

CAMPO	DESCRIPCIÓN	TIPO DE DATO
success	Indica si la operación fue exitosa	boolean
accounts	Lista de cuentas asociadas	array
accountNumber	Número de cuenta	string
accountType	Tipo de cuenta	string

Respuesta

```
{
  "GetMobileNumberAccountsResult": {
    "success": true,
    "accounts": [
      {
        "accountNumber": "string",
        "accountType": "string"
      }
    ]
  }
}
```

5.4 PRÉSTAMOS

Aplicación: Bankingly – GetLoan

Descripción: Consulta detalle de un préstamo.

Path: /Bankingly/Loans/GetLoan

Método: POST

Cabeceras de la petición

KEY	VALOR
Content-Type	application/json
Authorization	Bearer {token}

Datos de entrada

CAMPO	DESCRIPCIÓN	TIPO DE DATO
TenantId	Identificador de la institución	string
LoanNumber	Número del préstamo a consultar	string

Cuerpo de la petición

```
{
  "TenantId": "string",
  "LoanNumber": "string"
}
```

Datos de salida

CAMPO	DESCRIPCIÓN	TIPO DE DATO
success	Indica si la operación fue exitosa	boolean
loanNumber	Número del préstamo	string
amount	Monto original del préstamo	decimal
balance	Saldo pendiente	decimal
currency	Moneda	string
status	Estado del préstamo	string

Respuesta

```
{
  "GetLoanResult": {
    "success": true,
    "loanNumber": "string",
    "amount": "decimal",
    "balance": "decimal",
    "currency": "string",
    "status": "string"
  }
}
```

Aplicación: Bankingly – GetLoanFees

Descripción: Obtiene las cuotas de un préstamo.

Path: /Bankingly/Loans/GetLoanFees

Método: POST

Cabeceras de la petición

KEY	VALOR
Content-Type	application/json
Authorization	Bearer {token}

Datos de entrada

CAMPO	DESCRIPCIÓN	TIPO DE DATO
TenantId	Identificador de la institución	string
LoanNumber	Número del préstamo a consultar	string

Cuerpo de la petición

```
{
  "TenantId": "string",
  "LoanNumber": "string"
}
```

Datos de salida

CAMPO	DESCRIPCIÓN	TIPO DE DATO
success	Indica si la operación fue exitosa	boolean
fees	Lista de cuotas del préstamo	array
feeNumber	Número de la cuota	int
dueDate	Fecha de vencimiento	date
amount	Valor de la cuota	decimal
status	Estado de la cuota	string

Respuesta

```
{
  "GetLoanFeesResult": {
    "success": true,
    "fees": [
      {
        "feeNumber": 1,
        "dueDate": "yyyy-MM-dd",
        "amount": "decimal",
        "status": "string"
      }
    ]
  }
}
```

Aplicación: Bankingly – GetLoanPayments

Descripción: Obtiene los pagos realizados de un préstamo.

Path: /Bankingly/Loans/GetLoanPayments

Método: POST

Cabeceras de la petición

KEY	VALOR
Content-Type	application/json
Authorization	Bearer {token}

Datos de entrada

CAMPO	DESCRIPCIÓN	TIPO DE DATO
TenantId	Identificador de la institución	string
LoanNumber	Número del préstamo a consultar	string

Cuerpo de la petición

```
{
  "TenantId": "string",
  "LoanNumber": "string"
}
```

Datos de salida

CAMPO	DESCRIPCIÓN	TIPO DE DATO
success	Indica si la operación fue exitosa	boolean
payments	Lista de pagos realizados	array
date	Fecha del pago	date
amount	Monto pagado	decimal
receipt	Número de recibo	string

Respuesta

```
{
  "GetLoanPaymentsResult": {
    "success": true,
    "payments": [
      {
        "date": "yyyy-MM-dd",
        "amount": "decimal",
        "receipt": "string"
      }
    ]
  }
}
```

Aplicación: Bankingly – GetLoanRates

Descripción: Consulta las tasas de interés aplicables a un préstamo.

Path: /Bankingly/Loans/GetLoanRates

Método: POST

Cabeceras de la petición

KEY	VALOR
Content-Type	application/json
Authorization	Bearer {token}

Datos de entrada

CAMPO	DESCRIPCIÓN	TIPO DE DATO
TenantId	Identificador de la institución	string
LoanType	Tipo de préstamo	string

Cuerpo de la petición

```
{
  "TenantId": "string",
  "LoanType": "string"
}
```

Datos de salida

CAMPO	DESCRIPCIÓN	TIPO DE DATO
success	Indica si la operación fue exitosa	boolean
rates	Lista de tasas disponibles	array
rateType	Tipo de tasa (ej. fija, variable)	string
value	Valor de la tasa	decimal

Respuesta

```
{
  "GetLoanRatesResult": {
    "success": true,
    "rates": [
      {
        "rateType": "string",
        "value": "decimal"
      }
    ]
  }
}
```

5.5 TRANSACCIONES

Aplicación: Bankingly – InsertTransaction

Descripción: Registra una transacción en el sistema.

Path: /Bankingly/Transactions/InsertTransaction

Método: POST

Cabeceras de la petición

KEY	VALOR
Content-Type	application/json
Authorization	Bearer {token}

Datos de entrada

CAMPO	DESCRIPCIÓN	TIPO DE DATO
TenantId	Identificador de la institución	string
AccountOrigin	Número de la cuenta de origen	string
AccountDestination	Número de la cuenta de destino	string
Amount	Monto a transferir	decimal
Currency	Moneda en la que se realiza la transacción	string
Description	Descripción de la transacción	string

Cuerpo de la petición

```
{
  "TenantId": "string",
  "AccountOrigin": "string",
  "AccountDestination": "string",
  "Amount": "decimal",
  "Currency": "string",
  "Description": "string"
}
```

Datos de salida

CAMPO	DESCRIPCIÓN	TIPO DE DATO
success	Indica si la operación fue exitosa	boolean
transactionId	Identificador de la transacción	string
status	Estado de la transacción	string

Respuesta

```
{
  "InsertTransactionResult": {
    "success": true,
    "transactionId": "string",
    "status": "string"
  }
}
```


Aplicación: Bankingly – GetExchangeRateTransaction

Descripción: Consulta el tipo de cambio aplicado en una transacción.

Path: /Bankingly/Transactions/GetExchangeRateTransaction

Método: POST

Cabeceras de la petición

KEY	VALOR
Content-Type	application/json
Authorization	Bearer {token}

Datos de entrada

CAMPO	DESCRIPCIÓN	TIPO DE DATO
TenantId	Identificador de la institución	string
TransactionId	Identificador de la transacción	string

Cuerpo de la petición

```
{
  "TenantId": "string",
  "TransactionId": "string"
}
```

Datos de salida

CAMPO	DESCRIPCIÓN	TIPO DE DATO
success	Indica si fue exitosa	boolean
transactionId	Identificador de la transacción	string
exchangeRate	Tipo de cambio aplicado	decimal

Respuesta

```
{
  "GetExchangeRateTransactionResult": {
    "success": true,
    "transactionId": "string",
    "exchangeRate": "decimal"
  }
}
```

Aplicación: Bankingly – GetTransactionCost

Descripción: Consulta el costo asociado a una transacción.

Path: /Bankingly/Transactions/GetTransactionCost

Método: POST

Cabeceras de la petición

KEY	VALOR
Content-Type	application/json
Authorization	Bearer {token}

Datos de entrada

CAMPO	DESCRIPCIÓN	TIPO DE DATO
TenantId	Identificador de la institución	string
TransactionType	Tipo de transacción	string
Amount	Monto de la transacción	decimal

Cuerpo de la petición

```
{
  "TenantId": "string",
  "TransactionType": "string",
  "Amount": "decimal"
}
```

Datos de salida

CAMPO	DESCRIPCIÓN	TIPO DE DATO
success	Indica si fue exitosa	boolean
transactionType	Tipo de transacción consultada	string
cost	Costo de la transacción	decimal

Respuesta

```
{
  "GetTransactionCostResult": {
    "success": true,
    "transactionType": "string",
    "cost": "decimal"
  }
}
```

Aplicación: Bankingly – InsertMassiveTransaction

Descripción: Registra múltiples transacciones de forma masiva.

Path: /Bankingly/Transactions/InsertMassiveTransaction

Método: POST

Cabeceras de la petición

KEY	VALOR
Content-Type	application/json
Authorization	Bearer {token}

Datos de entrada

CAMPO	DESCRIPCIÓN	TIPO DE DATO
TenantId	Identificador de la institución	string
Transactions	Lista de transacciones a procesar	array

Cuerpo de la petición

```
{
  "TenantId": "string",
  "Transactions": [
```

```
{
  "AccountOrigin": "string",
  "AccountDestination": "string",
  "Amount": "decimal",
  "Currency": "string",
  "Description": "string"
}
```

Datos de salida

CAMPO	DESCRIPCIÓN	TIPO DE DATO
success	Indica si la operación fue exitosa	boolean
processedCount	Número de transacciones procesadas	int
failedCount	Número de transacciones fallidas	int

Respuesta

```
{
  "InsertMassiveTransactionResult": {
    "success": true,
    "processedCount": 10,
    "failedCount": 2
  }
}
```

5.6 AVAL BURÓ – Historial Crediticio

Aplicación: Aval Buró – Consulta de Historial Crediticio

Descripción: API que permite la consulta del historial crediticio de una persona, en el cual se realiza una consulta con el proveedor Aval Buró.

Path: /multicomm/Buro

Método: POST

Cabeceras de la petición

KEY	VALOR
aplicacioncliente	AvalBuro
nombreservicio	Consultar
aplicacionservidor	Aval

Datos de entrada

CAMPO	DESCRIPCIÓN	TIPO DE DATO
TipoIdentificacionSujeto	Tipo de identificación del documento del cliente (C, R, P, E, F).	string
IdentificacionSujeto	Número de identificación del cliente.	string

Cuerpo de la petición

```
{
  "tipoIdentificacionSujeto": "C",
```

```
"identificacionSujeto": "XXXXXXXXXXXX",  
}
```

Datos de salida

CAMPO	DESCRIPCIÓN	TIPO DE DATO
estadoTransaccion	True o False dependiendo si el mensaje fue enviado exitosamente	boolean
errorCodigo	Código de error generado al consumir el servicio (si aplica).	string
errorTipo	Tipo de error generado (si aplica).	string
errorDescripcion	Mensaje que explica el error o confirmación del resultado.	string
response	Mensaje tipo objeto del buro de datos	Object

Respuesta

```
{  
  "estadoTransaccion": true,  
  "errorCodigo": "",  
  "errorTipo": "",  
  "errorDescripcion": "Consulta ejecutada correctamente"  
  "response": {}  
}
```

5.7 REGISTRO CIVIL

Aplicación: Registro Civil – Consulta de Personas

Descripción: API que permite consultar la información de una persona en el Registro Civil. La información obtenida incluye datos generales, firma y fotografía.

Path: /lynko/RegistroCivil

Método: POST

Cabeceras de la petición

KEY	VALOR
aplicacioncliente	Denarius
nombreservicio	RegistroCivil
aplicacionservidor	RegistroCivil

Datos de entrada

CAMPO	DESCRIPCIÓN	TIPO DE DATO
NUI	Número de identificación de la persona.	string
IndividualDactilar	Código dactilar que aparece en la cédula (6 primeros dígitos de derecha a izquierda).	string

Cuerpo de la petición

```
{
  "RegistroCivil": {
    "NUI": "9999999999",
    "IndividualDactilar": "AXXXXA"
  }
}
```

Datos de salida

CAMPO	DESCRIPCIÓN	TIPO DE DATO
Calle	Calle principal del domicilio de la persona.	string
CodigoError	Código de error del Registro Civil (000 si no hay error).	string
CondicionCedulado	Condición de la persona (Ciudadano, Jubilado, etc.).	string
Conyuge	Nombres completos del cónyuge de la persona.	string
Domicilio	Dirección (Provincia/Cantón/Parroquia).	string
Error	Descripción del error, si existe.	string
EstadoCivil	Estado civil de la persona.	string
FechaCedulacion	Fecha de emisión de la cédula.	date
FechaFallecimiento	Fecha de deceso.	date
FechaMatrimonio	Fecha de matrimonio.	date
FechaNacimiento	Fecha de nacimiento.	date
Firma	Imagen base64 de la firma de la persona.	string
Fotografia	Imagen base64 de la foto de la persona.	string
Instrucción	Nivel de estudios de la persona.	string
LugarNacimiento	Lugar de nacimiento (Provincia/Cantón/Parroquia).	string
NUI	Número de identificación de la persona.	string
Nacionalidad	Nacionalidad de la persona.	string
Nombre	Nombres completos de la persona.	string
NumeroCasa	Número de la casa en el domicilio.	string
Profecion	Profesión u ocupación.	string
Sexo	Género de la persona.	string

5.8 MENSAJERÍA

Aplicación: Mensajería – Enviar Mensajes de Texto y Correos Electrónicos

Descripción: API que permite el envío de mensajes de texto (SMS) y correos electrónicos.

Path: /lynko/Mensajeria

Método: POST

Cabeceras de la petición

KEY	VALOR
aplicacioncliente	Denarius
nombreservicio	Mensaje
aplicacionservidor	Mensajería

Datos de entrada

CAMPO	DESCRIPCIÓN	TIPO DE DATO
Identificacion	Número de identificación del cliente (opcional, puede usarse si no se tiene actualizado celular/correo).	string
Email	Correo electrónico del cliente (obligatorio solo si es correo).	string
Subject	Concepto del mensaje (solo correos electrónicos).	string
Celular	Número de teléfono celular del cliente.	string
TipoMensaje	Tipo de mensaje a enviar (según tabla de anexos).	string
Valor	Valores a enviar en el SMS o correo (dependen del TipoMensaje).	string
Adjunto	Archivos adjuntos en correos (nombre.ext, base64).	string
EmailCC	Correos electrónicos en copia (solo en correos electrónicos).	string

Cuerpo de la petición

```
{
  "Mensaje": {
    "Identificacion": "xxxxxxxxxx",
    "Email": "xxxxx@xxxxx.xxx",
    "Subject": "Subject del correo electronico",
    "Celular": "Número de celular",
    "TipoMensaje": "xxxx",
    "Valor": "val1, val2, val3, ..., valN",
    "Adjunto": "nombre1.txt,base64,nombre2.txt,base64",
    "EmailCC": "xxxxx@xxxxx.xxx"
  }
}
```

Datos de salida

CAMPO	DESCRIPCIÓN	TIPO DE DATO
estadoTransaccion	Indica si el mensaje se envió exitosamente (True/False).	boolean
errorCodigo	Código de error (si aplica).	string
errorTipo	Tipo de error (si aplica).	string
errorDescripcion	Descripción del error o confirmación de envío.	string

Respuesta

```
{
```

```
"estadoTransaccion": true,  
"errorCodigo": "",  
"errorTipo": "",  
"errorDescripcion": "Mensaje enviado correctamente"  
}
```

5.9 COONECTA

Para el servicio de coonecta mantenemos tramas XML, y para todas estas tramas se tiene un header:

CAMPO	DESCRIPCIÓN	TIPO DE DATO
TraceId	Tipo de cuenta del cliente/ socio: • 10 Ahorros • 20 Corriente	string
EntityId	Número de cuenta del socio/cliente	string
EntityIdDest	Nombre del canal por el cual se ejecuta la transacción	string
ChannelId		
RealDate		
TerminalSeq		
MsgType		

Aplicación: Consulta de Clientes por número de Cuenta

Descripción: API que permite la consulta de clientes mediante su numero de cuenta.

Path: /coonecta/clienteNumCuenta

Método: POST

Datos de entrada

CAMPO	DESCRIPCIÓN	TIPO DE DATO
TipoCuenta	Tipo de cuenta del cliente/ socio: • 10 Ahorros • 20 Corriente	string
NroCuenta	Número de cuenta del socio/cliente	string
ChannelName	Nombre del canal por el cual se ejecuta la transacción	string

Cuerpo de la petición

```
{  
  "Mensaje": {  
    "Identificacion": "xxxxxxxxxx",  
    "Email": "xxxxx@xxxxx.xxx",  
    "Subject": "Subject del correo electronico",  
    "Celular": "Número de celular",  
    "TipoMensaje": "xxxx",  
  }  
}
```

```
"Valor": "val1, val2, val3, ...,valN",  
"Adjunto": "nombre1.txt,base64,nombre2.txt,base64",  
"EmailCC": "xxxxx@xxxxx.xxx"  
}  
}
```

Datos de salida

CAMPO	DESCRIPCIÓN	TIPO DE DATO
estadoTransaccion	Indica si el mensaje se envió exitosamente (True/False).	boolean
errorCodigo	Código de error (si aplica).	string
errorTipo	Tipo de error (si aplica).	string
errorDescripcion	Descripción del error o confirmación de envío.	string

Respuesta

```
{  
  "estadoTransaccion": true,  
  "errorCodigo": "",  
  "errorTipo": "",  
  "errorDescripcion": "Mensaje enviado correctamente"  
}
```